

The Problem:

// BAD: Duplicate components for each data type!

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}
```

```
interface Product {  
  id: number;  
  title: string;  
  price: number;  
}
```

// Separate component for users

```
function UserList({ users }: { users: User[] }) {  
  return (  
    <ul>  
      {users.map(user => (  
        <li key={user.id}>  
          {user.name} - {user.email}  
        </li>  
      ))}  
    </ul>  
  );  
}
```

// Duplicate component for products

```
function ProductList({ products }: { products: Product[] }) {  
  return (  
    <ul>  
      {products.map(product => (  
        <li key={product.id}>  
          {product.title} - ${product.price}  
        </li>  
      ))}  
    </ul>  
  );  
}
```

// 🤖 Need 10 different list components! Maintenance nightmare!

✅ Solution: Generic Component (One Component, All Types!):

// Single generic component works with ANY type!

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string | number;
  emptyMessage?: string;
}

function List<T>({
  items,
  renderItem,
  keyExtractor,
  emptyMessage = 'No items found'
}: ListProps<T>) {
  if (items.length === 0) {
    return <div className="empty">{emptyMessage}</div>;
  }

  return (
    <ul className="list">
      {items.map(item => (
        <li key={keyExtractor(item)}>
          {renderItem(item)}
        </li>
      ))}
    </ul>
  );
}
```

```
// ✅ Use with users – TypeScript knows 'item' is User!
```

```
<List
  items={users}
  keyExtractor={(user) => user.id}
  renderItem={(user) => (
    <div>
      <strong>{user.name}</strong>
      <span>{user.email}</span>
    </div>
  )}
/>
```

```
// ✅ Use with products – TypeScript knows 'item' is Product!
```

```
<List
  items={products}
  keyExtractor={(product) => product.id}
  renderItem={(product) => (
    <div>
      <h3>{product.title}</h3>
      <p>${product.price}</p>
    </div>
  )}
/>
```

```
// 🌟 One component, infinite types, full type safety!
```

🚀 Real-World Example – Generic Select/Dropdown:

// ✅ *Generic select works with any option type!*

```
interface SelectProps<T> {
  options: T[];
  value: T | null;
  onChange: (value: T) => void;
  getLabel: (option: T) => string;
  getValue: (option: T) => string | number;
  placeholder?: string;
}

function Select<T>({
  options,
  value,
  onChange,
  getLabel,
  getValue,
  placeholder = 'Select...'
}: SelectProps<T>) {
  return (
    <select
      value={value ? getValue(value) : ''}
      onChange={(e) => {
        const selected = options.find(
          opt => getValue(opt).toString() === e.target.value
        );
        if (selected) onChange(selected);
      }}
    >
      <option value="">{placeholder}</option>
      {options.map(option => (
        <option key={getValue(option)} value={getValue(option)}>
          {getLabel(option)}
        </option>
      ))}
    </select>
  );
}
```

//  Use with users

```
<Select
  options={users}
  value={selectedUser}
  onChange={setSelectedUser}
  getLabel={(user) => user.name}
  getValue={(user) => user.id}
  placeholder="Select user..."
/>
```

//  Use with products

```
<Select
  options={products}
  value={selectedProduct}
  onChange={setSelectedProduct}
  getLabel={(product) => product.title}
  getValue={(product) => product.id}
/>
```

//  Use with simple strings

```
<Select
  options={['apple', 'banana', 'orange']}
  value={selectedFruit}
  onChange={setSelectedFruit}
  getLabel={(fruit) => fruit}
  getValue={(fruit) => fruit}
/>
```

⚡ Generic Modal with Data:

// ✅ Modal that can display any data type!

```
interface ModalProps<T> {
  isOpen: boolean;
  onClose: () => void;
  data: T | null;
  renderContent: (data: T) => React.ReactNode;
  title?: string;
}

function Modal<T>({
  isOpen,
  onClose,
  data,
  renderContent,
  title
}: ModalProps<T>) {
  if (!isOpen || !data) return null;

  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal" onClick={(e) => e.stopPropagation()}>
        {title && <h2>{title}</h2>}
        <div className="modal-content">
          {renderContent(data)}
        </div>
        <button onClick={onClose}>Close</button>
      </div>
    </div>
  );
}
```


//  Use with user data

```
<Modal
  isOpen={isUserModalOpen}
  onClose={() => setIsUserModalOpen(false)}
  data={selectedUser}
  title="User Details"
  renderContent={(user) => (
    <div>
      <p><strong>Name:</strong> {user.name}</p>
      <p><strong>Email:</strong> {user.email}</p>
      <p><strong>Role:</strong> {user.role}</p>
    </div>
  )}
/>
```

//  Use with product data

```
<Modal
  isOpen={isProductModalOpen}
  onClose={() => setIsProductModalOpen(false)}
  data={selectedProduct}
  title="Product Details"
  renderContent={(product) => (
    <div>
      <h3>{product.name}</h3>
      <p>Price: ${product.price}</p>
      <p>Stock: {product.stock}</p>
    </div>
  )}
/>
```

🌟 Generic Hook:

// ✅ Generic hook for any paginated data!

```
interface PaginationResult<T> {
  data: T[];
  page: number;
  totalPages: number;
  loading: boolean;
  error: string | null;
  nextPage: () => void;
  prevPage: () => void;
  goToPage: (page: number) => void;
}

function usePagination<T>({
  fetchFn: (page: number) => Promise<{ data: T[]; totalPages: number }>,
  pageSize: number = 10
}): PaginationResult<T> {
  const [data, setData] = useState<T[]>([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    setLoading(true);
    setError(null);

    fetchFn(page)
      .then(result => {
        setData(result.data);
        setTotalPages(result.totalPages);
      })
      .catch(err => setError(err.message))
      .finally(() => setLoading(false));
  }, [page, fetchFn]);

  return {
    data,
    page,
    totalPages,
    loading,
    error,
    nextPage: () => setPage(p => Math.min(p + 1, totalPages)),
    prevPage: () => setPage(p => Math.max(p - 1, 1)),
    goToPage: (newPage) => setPage(Math.max(1, Math.min(newPage,
totalPages)))
  };
}
```


//  Use with users

```
const userPagination = usePagination<User>(
  (page) => fetch(`/api/users?page=${page}`).then(r => r.json())
);
```

//  Use with products

```
const productPagination = usePagination<Product>(
  (page) => fetch(`/api/products?page=${page}`).then(r => r.json())
);
```

Benefits:

//  Without Generics:

- // - Duplicate code for each type (10+ components)
- // - Hard to maintain
- // - No consistency
- // - Copy-paste errors

//  With Generics:

- // - One component for all types
- // - Easy to maintain
- // - Consistent behavior
- // - Full type safety
- // - Reusable across projects

// 90% less code duplication!